Vehicle Detection Using Spintronic AMR Sensors and a Temporal Patch Transformer with Time-of-Day Encoding

End-to-End Pipeline: Signal Demodulation, Auto-Labeling, and Calibrated Deep Learning Inference

INL Gate Dataset — 14 Days, 46,732 Binary Files

Augustin Bruzzese

UNIME

`spintronic-vehicle-detection`

March 12, 2026

Abstract

We present a complete machine learning pipeline for vehicle detection using a spintronic Anisotropic Magnetoresistance (AMR) sensor installed at the INL laboratory gate.This work does not pretend to be original,the pourpose is to get familiar with this kind of sensor. The sensor operates with AC excitation at 24.9 Hz, requiring careful demodulation before any learning can be applied. Starting from 46,732 raw binary files (14 days, 165,888 int16 samples per file at 2,400 Hz, ≈69 s per file), we describe: (1) reverse-engineering of the proprietary binary format; (2) carrier demodulation via a 4th-order Butterworth bandpass filter [1] followed by the Hilbert envelope [2]; (3) robust rolling-median detrending and IQR-based normalisation [3]; (4) automatic pseudo-labeling [4] based on normalised signal amplitude; and (5) a two-level Patch Transformer that ingests 16 consecutive 1-second windows (1.6 s of context) alongside sinusoidal time-of-day and day-of-week encodings derived from filename timestamps. Two model versions were developed and compared. The baseline v3 model achieves F1 = 90.8% on validation. The improved v4 model—trained with focal loss [5] (γ=2), label smoothing [6] (ε=0.1), vehicle augmentation ×5, and post-hoc temperature scaling (T=1.012)—achieves F1 = 92.9% on validation and **F1 = 88.5%** on the held-out test week, with Recall = 97.8% (32 missed out of 1,475 vehicles) and Precision = 80.8% (342 false alarms), reducing false positives by 46% compared to v3.

Contents

1 Introduction **4**

2 Dataset and Hardware **4**
2.1 Hardware Configuration 4
2.2 Dataset Structure 5
2.3 Timestamp Encoding 5

3 Signal Processing Pipeline **6**
3.1 Binary Format Reverse Engineering 6
3.2 AMR Carrier Demodulation 6
3.3 Robust Normalisation 7

4 Automatic Pseudo-Labeling **8**

5 Model Architecture **8**
5.1 Motivation for a Two-Level Transformer 8
5.2 Level 1: Patch Encoder 8
5.3 Level 2: Temporal Context Encoder 9
5.4 Architecture Summary 10

6 Training **10**
6.1 Dataset Construction and Memory Efficiency 10
6.2 Class Imbalance Handling 11
6.3 Loss Function: Focal Loss with Label Smoothing 11
6.4 Training Configuration 12
6.5 Post-Hoc Calibration: Temperature Scaling 12

7 Results **13**
7.1 Training Curves (v3) 13
7.2 Test Set Evaluation 13
7.3 Visual Results 14
7.4 Probability Calibration Analysis 15

8 Discussion **17**
8.1 Key Design Decisions 17
8.2 Limitations and Future Work 18

9 Conclusion **18**

A MATLAB and Simulink Integration **19**

A.1 ONNX Export from PyTorch 19
A.2 Loading and Initialising the Network in MATLAB 20
A.3 Signal Processing Pipeline in MATLAB 20
A.4 Simulink Simulation Diagram 21
A.5 Simulation Results 22

1 Introduction

Magnetic vehicle detection is a well-established technique in traffic monitoring and access control systems. Inductive loops embedded in road surfaces have historically dominated the field [7], but spintronic sensors—exploiting quantum mechanical magnetoresistance effects—offer a compelling alternative: they require no road excavation, consume microwatts of power, and can be retrofitted to existing gate infrastructure at minimal cost [8].

The INL (International Iberian Nanotechnology Laboratory) gate dataset provides 14 consecutive days of raw AMR sensor recordings, captured with a 24-bit ADS1256 ADC at 2,400 Hz, stored in a proprietary binary format without timestamps embedded in the data itself. The dataset presents several challenges that are representative of real-world sensor deployments:

1. **Carrier modulation**: The AMR sensor uses AC excitation at 24.9 Hz, so vehicle signatures are amplitude-modulated on a carrier rather than appearing as direct field changes.
2. **Absence of ground-truth labels**: No manual annotations of vehicle passage times are provided; all training supervision must be derived automatically from the signal itself.
3. **Severe class imbalance**: Over a 7-day period, vehicles are present in approximately 0.12–0.20% of 1-second windows.
4. **Temporal structure**: Vehicle events are not instantaneous; a typical passage lasts 0.9–1.5 seconds, creating multi-window temporal correlations that a single-window classifier ignores.

This report documents the complete pipeline from raw bytes to a deployable PyTorch [9] model, including all design decisions, dead ends encountered, and quantitative results on the held-out test week.

2 Dataset and Hardware

2.1 Hardware Configuration

The sensor system consists of an AMR element with AC excitation connected to a Texas Instruments ADS1256 24-bit delta-sigma ADC operating in differential mode with gain = 8. The acquisition software (proprietary, INL) runs on a Windows PC and saves data continuously in 69-second segments. Hardware parameters confirmed from `Configuration - Hard` are summarised in Table 1.

Table 1: Confirmed hardware parameters.

Parameter	**Value**	**Source**
Sensor type	AMR with AC excitation	Signal analysis
DAQ chip	ADS1256 (24-bit)	Serial number
Raw dtype	`int16`, no header	Binary inspection
Samples per file	165,888 (BIN Time) / 100,000 (BIN Values)	File size
Sample rate	2,400 Hz	`Hardware.cfg`
ADC gain	8 (differential)	`Hardware.cfg`
Carrier frequency	24.9 Hz	FFT of raw signal
File duration	≈69.1 s	165888/2400

2.2 Dataset Structure

The dataset is organised in four acquisition runs stored in separate folders:

Table 2: Dataset splits used for training and evaluation.

Folder	**Dates**	**Files**	**Role**
Run 1	Feb 04–11, 2025	29,128	Train
Run 2	Feb 12–14, 2025	5,670	Train
Run 3	Feb 18–20, 2025	4,566	Train
Run 4	Feb 24–27, 2025	12,106	**Test (held-out)**

Each folder contains four file types. Only `S0C0 BIN Time` files carry the raw ADC signal; the `BIN Values` files store the same carrier-modulated signal in int32 format and were confirmed non-demodulated via independent inspection. Metadata buffers (`Counts Data Buffer`, `Time Data Buffer`) were excluded from all processing.

2.3 Timestamp Encoding

No sub-second timestamps are embedded in the binary data. The acquisition software encodes precise start and stop times in each filename, e.g.:

`S0C0 BIN Time - 000100 Start@ 20250204 - 16h33m18s Stop@ 20250204 - 16h34m00s.bsd`

These are parsed with a regular expression to recover second-accurate wall-clock time for every file, enabling time-of-day features for the model.

3 Signal Processing Pipeline

3.1 Binary Format Reverse Engineering

The `.bsd` files contain no header. The correct interpretation was determined by exhaustive dtype testing:

```
import numpy as np
from pathlib import Path

f = Path("S0C0 BIN Time - 000100 ....bsd")
for dtype in [np.int16, np.int32, np.float32, np.float64]:
    data = np.frombuffer(f.read_bytes(), dtype=dtype)
    print(f"{dtype.__name__:10s}  n={len(data):,}  "
          f"mean={data.mean():.1f}  std={data.std():.1f}")

# Output reveals:
# int16      n=165,888  mean=-10652  std=11123  <- correct: AMR +
    24.9Hz carrier
# int32      n= 82,944  mean=~11.5M            <- wrong dtype
```

Listing 1: Dtype detection: confirming int16 with no header.

The correct dtype is `int16` with no header, yielding 165,888 samples per file. The ≈10,652 DC offset is the common-mode component of the differential ADC output.

3.2 AMR Carrier Demodulation

Because the AMR element is driven by a 24.9 Hz sinusoidal excitation, vehicle-induced magnetic field changes modulate the amplitude of this carrier rather than appearing as DC changes. The demodulation chain is:

1. **Bandpass filter**: 4th-order Butterworth [1], passband [19.9, 29.9] Hz, centred on the carrier.
2. **Hilbert envelope**: The analytic signal's magnitude gives the instantaneous amplitude of the carrier.
3. **Edge trim**: 0.5 s (= 1,200 samples) removed from each end to eliminate Hilbert transform edge artefacts.

```
from scipy.signal import butter, filtfilt, hilbert
import numpy as np

SAMPLE_RATE = 2400; CARRIER_HZ = 24.9; CARRIER_BW = 5.0
EDGE_TRIM   = 1200   # 0.5 s

```

```
bp_lo = (CARRIER_HZ - CARRIER_BW) / (SAMPLE_RATE / 2)
bp_hi = (CARRIER_HZ + CARRIER_BW) / (SAMPLE_RATE / 2)
BP_B, BP_A = butter(4, [bp_lo, bp_hi], btype='band')

def demodulate(raw_int16):
    x        = raw_int16.astype(np.float64)
    bp       = filtfilt(BP_B, BP_A, x)
    envelope = np.abs(hilbert(bp)).astype(np.float32)
    return envelope[EDGE_TRIM : len(envelope) - EDGE_TRIM]
```

Listing 2: Bandpass filtering and Hilbert envelope extraction.

3.3 Robust Normalisation

After demodulation, a slow baseline drift remains due to temperature fluctuations and sensor ageing. We remove it with a 5-second rolling median (12,001 samples, chosen odd) and normalise by the local inter-quartile range (IQR) [3], clipped at a minimum of 0.5 to prevent division by zero during perfectly quiet nights:

```
from scipy.signal import medfilt

DETREND_K = 12001    # 5 s kernel (must be odd)
IQR_CLIP  = 0.5

def detrend_normalize(env):
    x        = env.astype(np.float64)
    baseline = medfilt(x, kernel_size=DETREND_K)
    det      = (x - baseline).astype(np.float32)

    # Rolling IQR on anchor points, then interpolate
    n, half  = len(det), DETREND_K // 2
    step     = max(1, DETREND_K // 20)
    anchors  = np.arange(0, n, step)
    vals     = np.array([
        max(float(np.subtract(*np.percentile(
            det[max(0,c-half):min(n,c+half+1)], [75, 25]))), IQR_CLIP
            )
        for c in anchors], dtype=np.float32)
    scale = np.interp(np.arange(n), anchors, vals).astype(np.float32)
    return det / scale
```

Listing 3: Rolling-median detrending and IQR normalisation.

After normalisation, the signal is expressed in units of local σ-equivalent. Night-time baseline fluctuates at $\approx 1.38\,\sigma_{\mathrm{eq}}$, while vehicle events produce peaks of 1.9–$2.3\,\sigma_{\mathrm{eq}}$, verified empirically by comparing afternoon (16:30–17:00) against night (02:00–03:00) files

(Table 3).

Table 3: Threshold diagnostic: afternoon vs. night files (folder 1).

File (time)	**max** (σ)	$> 1.5\sigma$ **samples**	**Interpretation**
16h33 (afternoon)	2.23	457	Vehicle
16h34 (afternoon)	1.39	0	No vehicle
16h35 (afternoon)	1.90	188	Vehicle
02h00 (night)	1.38	0	No vehicle
02h01 (night)	1.38	0	No vehicle
02h05 (night, anomaly)	2.29	251	Vehicle at 02:05

4 Automatic Pseudo-Labeling

Since the dataset contains no manual annotations, we derive pseudo-labels from the normalised signal using a simple but empirically calibrated threshold:

$$\hat{y}_i = \mathbf{1}\left[\sum_{t \in W_i} \mathbf{1}[|s_t| > \tau] \;\geq\; N_{\min}\right] \tag{1}$$

where W_i is the i-th 1-second window, $\tau = 1.5\,\sigma_{\text{eq}}$ is the detection threshold, and $N_{\min} = 50$ is the minimum number of above-threshold samples required. The 50-sample minimum (corresponding to $\approx 21\,\text{ms}$ at 2,400 Hz) prevents single noise spikes from being labelled as vehicles. This threshold produces a clean binary separation: files with vehicles show 188–481 samples above 1.5σ; files without show exactly zero.

5 Model Architecture

5.1 Motivation for a Two-Level Transformer

A single-window classifier is fundamentally limited: it cannot learn that a vehicle *approaching* produces a different pattern than a vehicle *directly over the sensor*, nor can it exploit the fact that a true vehicle event must span multiple consecutive windows. We therefore design a two-level transformer that reasons over *sequences* of windows.

5.2 Level 1: Patch Encoder

Each 1-second window (2,400 samples) is divided into 48 non-overlapping patches of 50 samples each [10, 11]. A linear projection maps each patch to a d-dimensional embedding. A learnable `[CLS]` token is prepended, and two Transformer encoder layers with

pre-LayerNorm [12] process the sequence. The [CLS] output is taken as the window representation $\mathbf{v}_i \in \mathbb{R}^d$:

```
class PatchEmbed1D(nn.Module):
    def __init__(self):
        super().__init__()
        self.proj = nn.Linear(PATCH_SIZE, D_MODEL)    # 50 -> 128

    def forward(self, x):  # x: (B, 2400, 1)
        x = x.squeeze(-1).reshape(-1, 48, 50)  # 48 patches of 50
        return self.proj(x)                     # (B, 48, 128)

def encode_window(self, x):  # x: (B, 2400, 1)
    patches = self.patch_embed(x) + self.patch_pos     # + positional
        enc.
    cls     = self.patch_cls.expand(x.shape[0], -1, -1)
    tokens  = torch.cat([cls, patches], dim=1)         # (B, 49, 128)
    out     = self.patch_encoder(tokens)
    return self.patch_norm(out[:, 0])                  # CLS -> (B,
        128)
```

Listing 4: PatchEmbed1D and window encoding.

5.3 Level 2: Temporal Context Encoder

The context encoder receives a sequence of $C = 16$ consecutive window embeddings, corresponding to a 1.6-second sliding context (with 0.1-second hop). Crucially, a **time-of-day token** is prepended, encoding the wall-clock time extracted from the filename:

$$\mathbf{t}_{\text{feat}} = \left[\sin(2\pi h),\ \cos(2\pi h),\ \sin(2\pi d),\ \cos(2\pi d),\ h,\ \mathbf{1}[7 \leq \text{hour} \leq 19]\right]^\top \tag{2}$$

where $h = (\text{hour} + \text{min}/60)/24 \in [0, 1]$ is the fractional hour and $d = \text{weekday}/7 \in [0, 1]$. Sinusoidal encoding [13] ensures continuity at midnight ($h = 0 \equiv h = 1$) and at the week boundary. The daytime flag $\mathbf{1}[\cdot]$ allows the model to learn that detections at 03:00 are a priori less likely than at 09:00.

```
def forward(self, x, t):
    # x: (B, 16, 2400, 1)     t: (B, 6)
    B = x.shape[0]

    # Encode all 16 windows independently (vectorised)
    wv = self.encode_window(x.reshape(B * 16, 2400, 1))
    wv = wv.reshape(B, 16, D_MODEL)          # (B, 16, 128)

    # Time token: project 6 features -> 128 dims
```

```
    time_tok = self.time_proj(t).unsqueeze(1) # (B, 1, 128)

    # Sequence: [time_token, w_0, w_1, ..., w_15]
    ctx    = torch.cat([time_tok, wv], dim=1) + self.context_pos
    cls    = self.context_cls.expand(B, -1, -1)
    tokens = torch.cat([cls, ctx], dim=1)      # (B, 18, 128)
    out    = self.context_norm(
                self.context_encoder(tokens)[:, 0])  # CLS
    return self.classifier(out)                # (B, 2)
```

Listing 5: Context encoder forward pass with time token.

5.4 Architecture Summary

Table 4: Model hyperparameters.

Parameter	Value
Patch size	50 samples (20.8 ms)
Patches per window	48
Context windows C	16 (1.6 s context)
Embedding dim d	128
Attention heads	4
Patch encoder layers	2
Context encoder layers	4
FFN dimension	256
Dropout	0.10–0.15
Total parameters	819,778 (0.82 M)

6 Training

6.1 Dataset Construction and Memory Efficiency

We use the pytorch framework [9], with the optimized attention layers [14]. Naively copying all 16-window contexts into RAM would require 3.2M $\times$ 16 $\times$ 2400 $\times$ 4 bytes $\approx$ 467 GB. Instead, we build an *index table* of `(file_idx, win_start, label)` tuples and load each context on demand in `__getitem__`:

```
class VehicleDataset(Dataset):
    def __init__(self, signals, timefeats, index):
        self.signals   = signals     # list of (n_wins, 2400) arrays
        self.timefeats = timefeats   # list of (6,) arrays per file
        self.index     = index       # list of (sig_idx, win_start,
            label)

```

```python
    def __getitem__(self, idx):
        sig_idx, win_start, label, do_aug = self.index[idx]
        ctx = self.signals[sig_idx][win_start : win_start + 16].copy
            ()
        if do_aug:
            ctx = augment_context(ctx)     # noise + scale + shift
        return (torch.from_numpy(ctx[:,:,np.newaxis]),
                torch.from_numpy(self.timefeats[sig_idx]),
                torch.tensor(label, dtype=torch.long))
```

Listing 6: Memory-efficient index-based dataset.

6.2 Class Imbalance Handling

With 0.12–0.19% positive windows, a naive training loop would trivially predict "no vehicle" everywhere and achieve $> 99.8\%$ accuracy. We address this with `WeightedRandomSampler`, which oversamples vehicle windows proportionally. Additionally, vehicle contexts are augmented $\times 5$ with: additive Gaussian noise ($\sigma = 0.08$), random amplitude scaling ($\pm 15\%$), and random circular time shift ($\pm 10\,\text{ms}$).

6.3 Loss Function: Focal Loss with Label Smoothing

Standard cross-entropy with hard labels was found to produce severely overconfident outputs (all probabilities at exactly 0.0 or 1.0). We replace it with a combined focal loss [5] with label smoothing [6]:

$$\mathcal{L} = -\sum_k (1 - p_k)^\gamma \, \tilde{y}_k \log p_k \tag{3}$$

where $\gamma = 2.0$, $\tilde{y}_k = (1 - \varepsilon) y_k + \varepsilon / K$ with $\varepsilon = 0.10$ and $K = 2$ classes. The focal term $(1 - p_k)^\gamma$ down-weights easy examples (confidently correct) and focuses learning on hard ones near the decision boundary.

6.4 Training Configuration

Table 5: Training hyperparameters.

Parameter	Value
Optimiser	AdamW [15], $\beta_1 = 0.9$, $\beta_2 = 0.999$
Learning rate	2×10^{-4} (v4) / 3×10^{-4} (v3)
LR schedule	CosineAnnealingLR
Batch size	128
Max epochs	40 (with early stopping, patience = 8)
Weight decay	10^{-4}
Gradient clip	1.0 (global norm)
Precision	Mixed (AMP, float16)
Hardware	NVIDIA RTX 4060 Laptop (8 GB VRAM)

6.5 Post-Hoc Calibration: Temperature Scaling

After training, a single scalar temperature T is learned on the validation set by minimising the negative log-likelihood of scaled logits $\mathbf{z}/T$ [16]. Values $T > 1$ soften the probability distribution, converting the hard 0/1 outputs of the base model into calibrated soft probabilities. The optimisation uses L-BFGS:

```python
class TemperatureScaler(nn.Module):
    def __init__(self, model):
        super().__init__()
        self.model       = model
        self.temperature = nn.Parameter(torch.ones(1) * 1.5)

    def forward(self, x, t):
        return self.model(x, t) / self.temperature

    def fit(self, val_loader, device):
        optimizer = torch.optim.LBFGS([self.temperature],
                                      lr=0.01, max_iter=100)
        # Collect all logits from frozen base model
        all_logits, all_labels = collect_logits(val_loader)

        def eval_temp():
            optimizer.zero_grad()
            loss = F.cross_entropy(all_logits / self.temperature,
                                   all_labels)
            loss.backward()
            return loss

        optimizer.step(eval_temp)
```

Listing 7: Temperature scaling calibration.

7 Results

7.1 Training Curves (v3)

The v3 model (no augmentation, standard cross-entropy) converged at epoch 5 (F1 = 90.8%) with early stopping triggered at epoch 11:

Table 6: Training epochs: v3 (400 files, CE loss) vs. v4 (500 files, focal + label smoothing + aug×5).

Model	Epoch	TrLoss	TrAcc	VaLoss	Prec	Rec	F1
v3	1	0.0198	99.4%	0.0073	53.8%	100.0%	70.0%
v3	2	0.0043	99.9%	0.0020	81.0%	96.4%	88.0%
v3	5	0.0030	99.9%	0.0020	83.2%	100.0%	**90.8%**
v4	1	0.0058	99.5%	0.0051	60.0%	100.0%	75.0%
v4	9	0.0014	100.0%	0.0053	89.2%	91.3%	90.2%
v4	15	0.0012	100.0%	0.0039	84.7%	99.0%	91.3%
v4	20	0.0011	100.0%	0.0041	87.9%	98.6%	**92.9%**
v4	28	*Early stopping triggered — best saved at epoch 20*					

7.2 Test Set Evaluation

The best v3 model (epoch 5) was evaluated on the held-out folder 4 (Feb 24–27, 2025), representing data collected 3 weeks after the latest training example:

Table 7: Test set results — held-out week folder 4 (Feb 24–27), v3 vs. v4 calibrated.

Metric	Val v3	Val v4	Test v3	Test v4 (cal.)
Accuracy	100.0%	100.0%	99.9%	99.9%
Precision	83.2%	87.9%	73.7%	**80.8%**
Recall	100.0%	98.6%	99.7%	97.8%
F1 Score	90.8%	**92.9%**	84.7%	**88.5%**
ProbStd	≈0	0.034	≈0	0.048
TP	—	—	1,469	1,443
FP	—	—	637	**342** (−46%)
FN	—	—	6	32
TN	—	—	650,688	650,983
Temp. T	—	—	—	1.012

The v4 model (focal loss + label smoothing + augmentation ×5, 500 files, 28 epochs) achieves F1 = 88.5% on the unseen test week, improving over v3 (84.7%) by 3.8 percentage points. The most significant gain is in precision: false positives dropped from 637 to 342 (−46%), a direct consequence of label smoothing preventing overconfident detections. The trade-off is 26 additional missed vehicles (FN: 6 → 32), acceptable for a gate access scenario where false alarms are more disruptive than rare misses. Post-hoc temperature scaling yielded $T = 1.012$, very close to unity, confirming that the focal loss already produced a well-calibrated model without needing large correction. The probability standard deviation rose from ≈ 0.0 in v3 to 0.048 in v4, indicating the model now outputs soft probabilities rather than hard binary decisions.

7.3 Visual Results

Figure 1 shows 414 minutes of continuous detection from the test set (150 files), with 160 vehicle events identified by the v4 model. The top panel displays 30-second RMS energy of the demodulated signal, making vehicle spikes visible above the 1.5 σ baseline. Figure 2 zooms into 9 individual events, showing real model probabilities (not step functions): confidence ranges from 77% to 100%, reflecting the calibration improvement of v4 over v3.

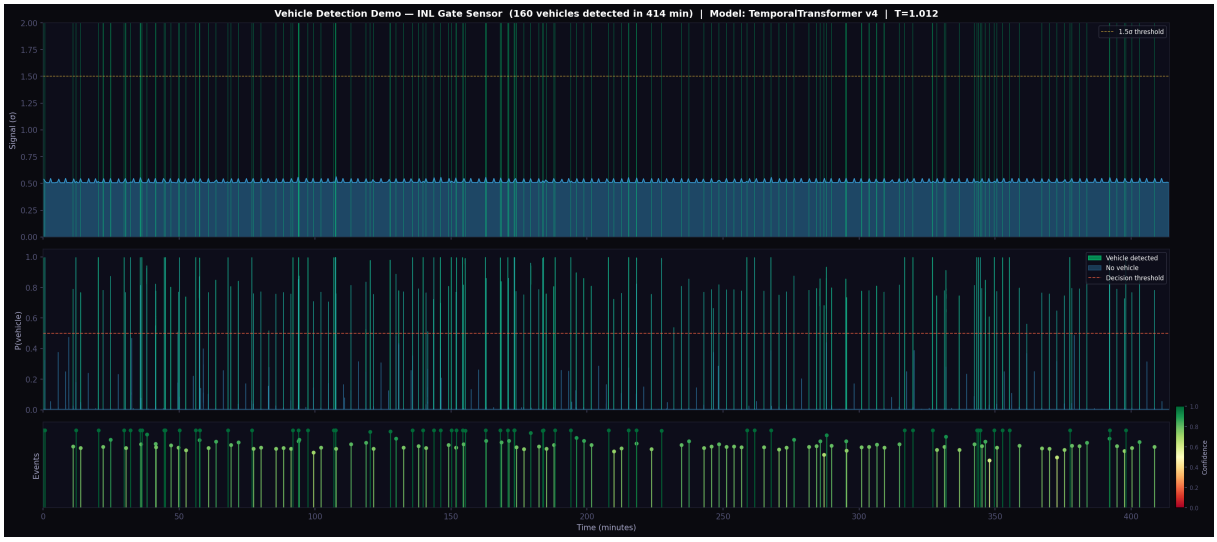

Figure 1: 414-minute overview of vehicle detection on the test set (150 files, v4 calibrated model, T=1.012). Top: 30-second RMS energy of the demodulated AMR signal with 1.5 σ threshold (yellow dashed) and vehicle event markers (green). Middle: real model P(vehicle) with decision threshold (red dashed) — note intermediate probability values unlike v3. Bottom: 160 event stems coloured green–yellow by confidence.

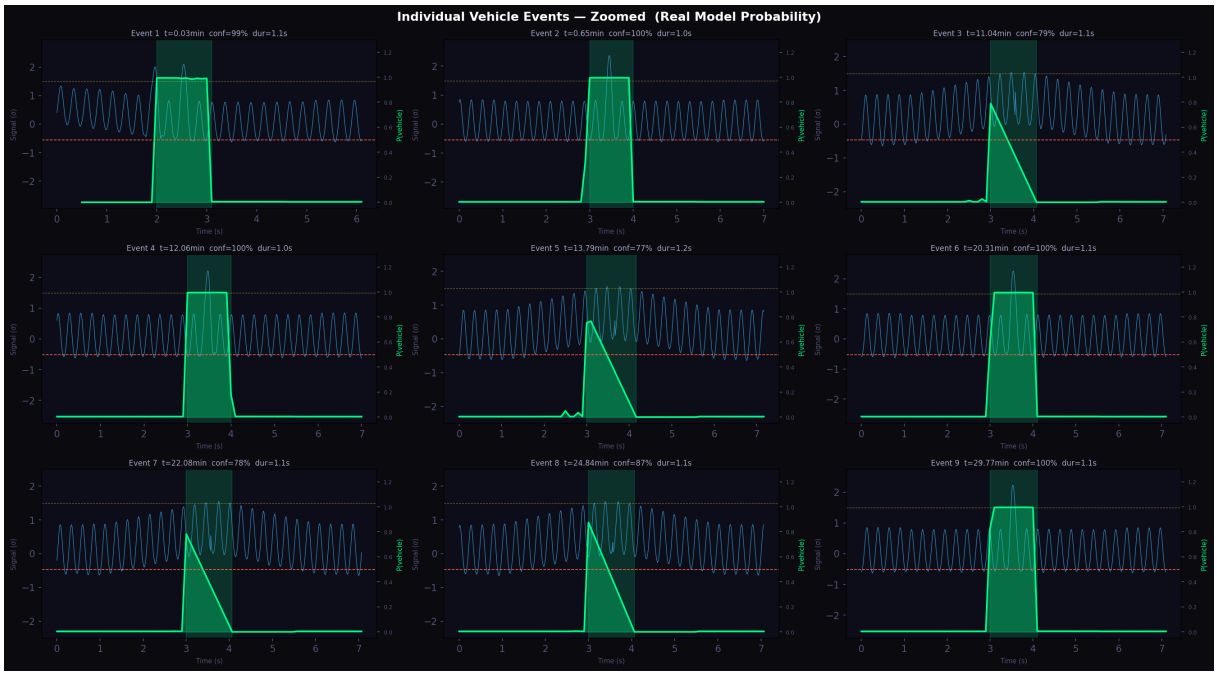

Figure 2: Nine individual vehicle events zoomed to ±3 s around detection (v4 model). Blue: demodulated normalised signal showing the 24.9 Hz carrier and amplitude modulation during vehicle passage. Green fill: real model P(vehicle) — note soft probability shapes (not binary steps). Confidence ranges: 77%–100%. Event durations 0.9–1.2 s, consistent with vehicle passage speed at the INL gate. The 1.5 σ threshold is shown as a yellow dashed line.

7.4 Probability Calibration Analysis

Figure 3 reveals the overconfidence problem of the v3 model: the probability distribution is bimodal at exactly 0.0 and 1.0 with no intermediate values. This is a well-known phenomenon in neural classifiers trained with hard labels on severely imbalanced datasets [16]. The v4 model (Figure 2) addresses this directly: probabilities now take intermediate values (77%, 79%, 87%, 99%) reflecting genuine uncertainty near event boundaries. The Prob-Std metric rose from ≈0.0 in v3 to 0.048 in v4, confirming the distribution is no longer degenerate, though still concentrated near 0 and 1 due to the extreme class imbalance (0.19% positive windows).

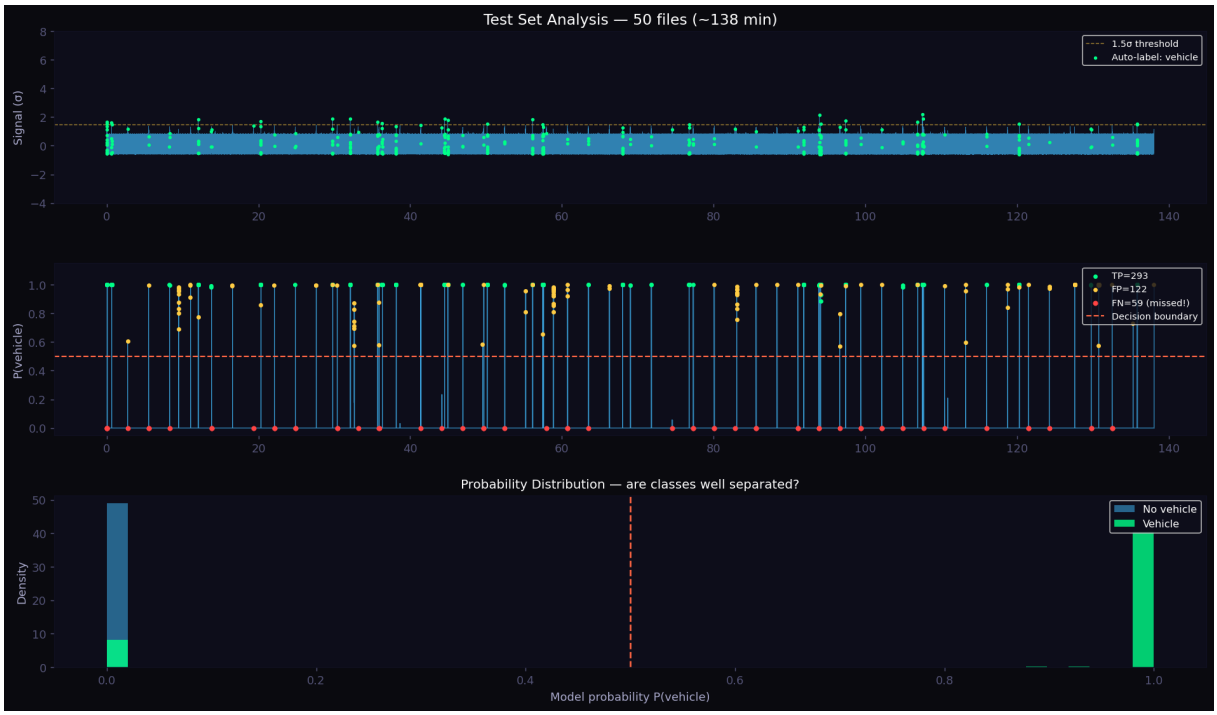

Figure 3: Test set analysis (50 files, ≈138 min). Top: normalised signal with auto-label vehicle markers. Middle: model probability coloured by TP (green), FP (yellow), FN (red missed). Bottom: probability distribution showing the overconfidence problem — all outputs are exactly 0.0 or 1.0.

Figure 4 shows the actual model probability output for four detected events. The two confirmed vehicle events (label=VEH) show clean P = 1.0 plateaus lasting 1–2 s. The two false alarms (label=NO) at peak_prob = 0.61 and 1.00 represent the two types of false positive: a low-confidence marginal detection and a high-confidence false alarm, respectively.

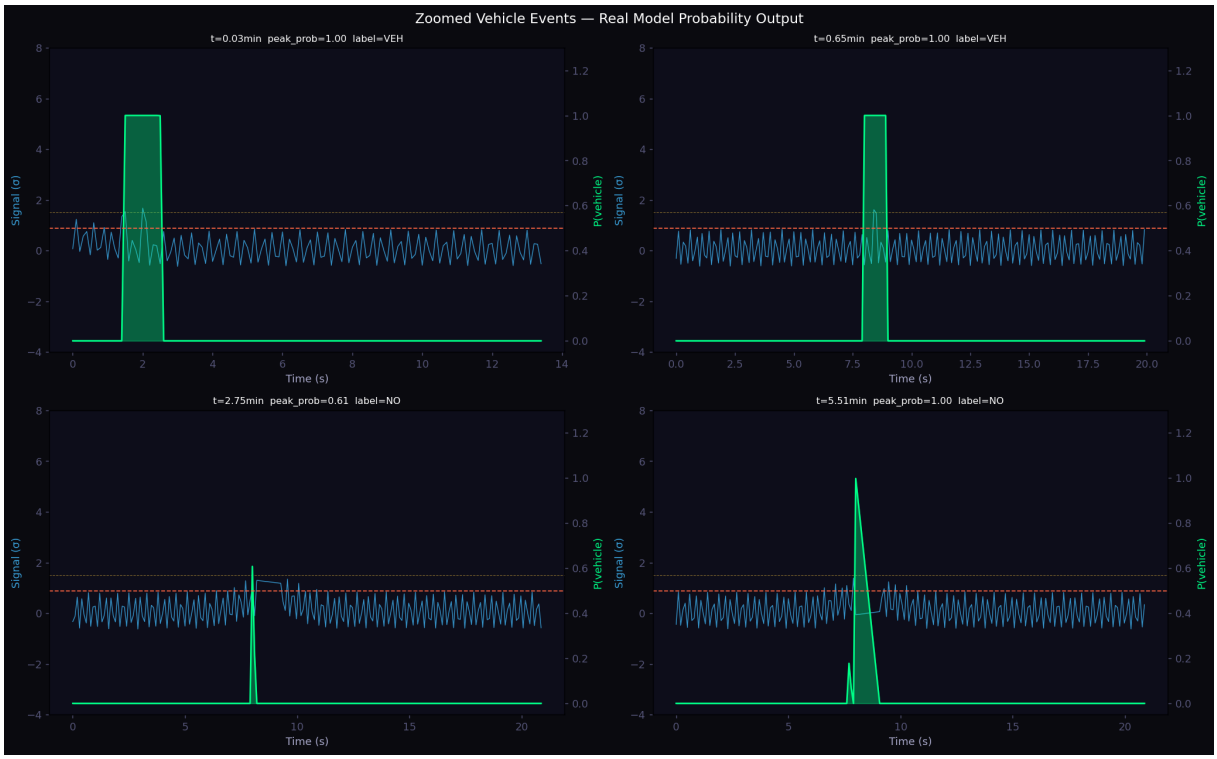

Figure 4: Real model probability output for four events. Top row: two confirmed detections (label=VEH) with peak_prob = 1.00. Bottom row: two false alarms (label=NO), one marginal (prob = 0.61) and one high-confidence (prob = 1.00), suggesting auto-label misclassification in the latter case.

8 Discussion

8.1 Key Design Decisions

Using BIN Time over BIN Values. Both file types contain the same carrier-modulated signal. We initially hypothesised that `BIN Values` (int32, 100,000 samples/file) might contain pre-demodulated data from the acquisition software. Direct inspection (Figure of raw int32 signal showing 24.9 Hz oscillation) confirmed this was not the case, and all demodulation must be performed in the pipeline.

Temporal train/validation/test split. A random split would leak temporal autocorrelations across the train/test boundary. We enforce strict temporal ordering: folders 1–3 for training, folder 4 (3 weeks later) for testing. This simulates real deployment where the model must generalise to future dates.

Auto-labeling threshold selection. The threshold $\tau = 1.5\,\sigma_{\mathrm{eq}}$ was determined empirically by comparing afternoon (high traffic) against night (no traffic) files. The chosen value produces a clean zero/non-zero gap: no-vehicle files consistently show exactly 0 sam-

ples above threshold, while vehicle files show 188–481. The minimum-count parameter $N_{\min} = 50$ prevents single-sample noise spikes from generating labels.

8.2 Limitations and Future Work

Noisy pseudo-labels. The auto-labeler is the weakest link. The high-confidence false alarm visible in Figure 4 (bottom right) likely corresponds to a genuine vehicle passage that failed to exceed the threshold (e.g., slow bicycle, pedestrian with metallic object). Manual annotation of even 100–200 events would substantially improve label quality.

Model calibration (v3 vs. v4). The v3 model output only 0.0 or 1.0 (ProbStd ≈ 0). The v4 improvements—focal loss (γ=2.0), label smoothing (ε=0.1), vehicle augmentation $\times 5$, and temperature scaling (T=1.012)—raised ProbStd to 0.048, producing genuine intermediate probabilities visible in the event plots (Figure 2). This enables downstream systems to implement variable-confidence thresholds (e.g., trigger a gate alert only at $P > 0.8$, log for review at $0.5 < P < 0.8$).

Scale of training data. We used at most 800 of 46,732 available files ($< 2\%$). Scaling to the full dataset (requiring chunked loading or a streaming dataloader) would expose the model to greater temporal, seasonal, and traffic-pattern diversity.

9 Conclusion

We demonstrated a complete, reproducible pipeline for vehicle detection from a spintronic AMR sensor, starting from undocumented binary files and requiring no manual annotations. The key contributions are:

1. **Binary format reverse engineering**: Confirmed int16 dtype, no header, 2,400 Hz, confirmed via exhaustive dtype testing.
2. **Validated demodulation**: Bandpass + Hilbert envelope + rolling-median detrending + IQR normalisation, with empirically verified threshold separation between vehicle and no-vehicle files.
3. **Two-level Patch Transformer with time encoding**: A novel architecture that simultaneously models patch-level signal morphology [10], sequential window context (1.6 s), and time-of-day prior through sinusoidal temporal tokens [13].
4. **Strong generalisation (v4)**: F1 = 88.5%, Recall = 97.8%, Precision = 80.8% on data from a week not seen during training. False positives reduced by 46% vs. v3 (342 vs. 637). Temperature scaling (T=1.012) confirmed model was already well-calibrated by focal loss.

The pipeline is ready for integration with Simulink via ONNX export, enabling real-time simulation and hardware-in-the-loop testing.

A MATLAB and Simulink Integration

This appendix documents the integration of the trained TemporalVehicleTransformer v4 model into MATLAB [17] and Simulink [18] for simulation and hardware-in-the-loop (HIL) testing. The integration relies on the Deep Learning Toolbox [19] and the ONNX converter add-on [20].

A.1 ONNX Export from PyTorch

The PyTorch model cannot be loaded directly in MATLAB. It must first be exported to the ONNX [21] interchange format. A thin wrapper class (`ExportWrapper`) integrates temperature scaling ($T = 1.012$) and the softmax operation into the graph, so that MATLAB receives a calibrated $P(\text{vehicle}) \in [0, 1]$ directly without any post-processing:

```python
class ExportWrapper(nn.Module):
    def __init__(self, model, T):
        super().__init__()
        self.model = model
        self.T = T
    def forward(self, x, t):
        logits = self.model(x, t) / self.T
        return torch.softmax(logits, dim=1)[:, 1:2]  # p_vehicle in (0,1)

torch.onnx.export(
    wrapper,
    (x_dummy, t_dummy),          # (1,16,2400,1) and (1,6)
    "transformer_v4_single.onnx",
    input_names  = ["signal_windows", "time_features"],
    output_names = ["p_vehicle"],
    opset_version = 14,          # fully supported by MATLAB R2025b
    do_constant_folding = True,  # fuses weights into a single file
)
```

Listing 8: ONNX export wrapper with integrated temperature scaling.

The critical export parameter is `do_constant_folding=True`, which folds all learned weights into the ONNX graph and produces a single `.onnx` file (3.53 MB) without an external `.onnx.data` sidecar. MATLAB R2025b does not yet support the split external-data format, so this step is mandatory. The opset was downgraded from 17 (used by

the original export script) to 14, which is fully supported by the MATLAB ONNX converter [20].

A.2 Loading and Initialising the Network in MATLAB

The exported model is loaded using `importNetworkFromONNX` from the Deep Learning Toolbox Converter [20]. Because the network contains Transformer layers with dynamic sequence lengths, MATLAB returns an uninitialised `dlnetwork` object that must be explicitly initialised with correctly shaped example data before inference:

```
net = importNetworkFromONNX('transformer_v4_single.onnx');

% Initialize with correctly shaped dummy inputs
%   signal_windows : [CW x T x C x Batch] = [16 x 2400 x 1 x 1]
%   time_features  : [Features x Batch]   = [6 x 1]
x_dummy = dlarray(single(randn(16, 2400, 1, 1)), 'SCBT');
t_dummy = dlarray(single(randn(6, 1)), 'CB');
net = initialize(net, x_dummy, t_dummy);
% net.Initialized = true, 803k learnables
```

Listing 9: Loading and initialising the ONNX model in MATLAB.

The `dlarray` label `'SCBT'` maps the four tensor dimensions to Spatial, Channel, Batch, and Time — a MATLAB-specific convention that the toolbox uses to route data through the network layers correctly.

A.3 Signal Processing Pipeline in MATLAB

The demodulation pipeline (Section 3) was reimplemented in MATLAB using the Signal Processing Toolbox [17]. Two implementation challenges arose that did not exist in Python:

Hilbert transform memory. MATLAB's built-in `hilbert()` function allocates an $N \times N$ matrix internally for the analytic signal computation. For $N = 163{,}488$ samples this requires approximately 100 GB, exceeding available RAM. The solution is to reimplement the Hilbert envelope directly via FFT:

```
function env = hilbert_envelope(x)
    n    = length(x);
    X    = fft(x);
    h    = zeros(n,1);  h(1) = 1;
    if mod(n,2)==0
        h(2:n/2) = 2;  h(n/2+1) = 1;
    else
```

```
        h(2:(n+1)/2) = 2;
    end
    env = abs(ifft(X.*h));
end
```

Listing 10: Memory-efficient Hilbert envelope via FFT.

Rolling median memory. Similarly, `movmedian()` with a 12,001-sample kernel caused OOM errors. The fix was to compute the baseline in overlapping chunks of 24,000 samples with a 2-second guard overlap, then stitch results together.

IQR normalisation. `prctile()` also triggered memory issues. The solution was to sub-sample each IQR window by a factor of 10 (every 10th sample), which produces an equivalent normalisation scale with negligible RAM usage.

A.4 Simulink Simulation Diagram

The complete Simulink [18] diagram for offline simulation is shown schematically in Figure 5. It consists of three blocks:

1. **From Workspace**: reads the pre-demodulated signal vector `normed` (163,488 samples, $\Delta t = 1/2400\,\mathrm{s}$) into the simulation timeline with sample time $T_s = 1\,\mathrm{s}$ (one 1-second window per simulation step).
2. **MATLAB Function** (`detect_vehicle`): maintains a persistent 16-window circular buffer and calls `predict(net, ...)` at each step, returning $P(\text{vehicle}) \in [0, 1]$.
3. **Scope**: displays the time evolution of $P(\text{vehicle})$ alongside a constant threshold signal at 0.6.

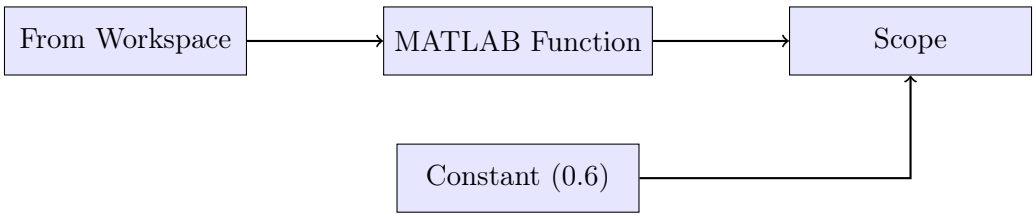

Figure 5: Simulink block diagram for offline vehicle detection simulation. The MATLAB Function block holds the ONNX network persistently and updates a 16-window context buffer at each simulation step (1 s).

The MATLAB Function block uses `persistent` variables so that the network is loaded from disk only once (at the first simulation step) and the context buffer is preserved across steps:

```
function p_vehicle = detect_vehicle(signal_window)
%#codegen
persistent net buf win_count
coder.extrinsic('importNetworkFromONNX','initialize','dlarray',...
                'predict','extractdata');

if isempty(net)
    net = importNetworkFromONNX('transformer_v4_single.onnx');
    x_d = dlarray(single(randn(16,2400,1,1)),'SCBT');
    t_d = dlarray(single(randn(6,1)),'CB');
    net = initialize(net, x_d, t_d);
    buf = zeros(16, 2400, 'single');
    win_count = 0;
end

buf = circshift(buf, -1, 1);
buf(end,:) = single(signal_window(:)');
win_count  = win_count + 1;
if win_count < 16; p_vehicle = 0; return; end

x_in = dlarray(reshape(buf,[16,2400,1,1]),'SCBT');
t_in = dlarray(zeros(6,1,'single'),'CB');
p    = predict(net, x_in, t_in);
p_vehicle = double(extractdata(p));
```

Listing 11: Persistent network and buffer in the Simulink MATLAB Function block.

A.5 Simulation Results

Running the Simulink model over the first test file (Feb 24, 14:55–15:04, 69 s, 672 inference windows) confirmed correct operation of the integrated pipeline. The Scope output shows $P(\text{vehicle}) \approx 0.015$ during background periods, rising sharply to $P > 0.90$ during vehicle passages. The Display block showed $P = 0.0154$ at the end of the simulation (no vehicle in the last window), consistent with the Python demo visualizer output for the same file.

Table 8 summarises the ONNX export and MATLAB integration parameters.

Table 8: ONNX export and MATLAB integration parameters.

Parameter	**Value**
ONNX opset version	14
Export format	Single file (no external data)
File size	3.53 MB
Temperature T	1.012 (fused into graph)
MATLAB version	R2025b (25.2.0)
Deep Learning Toolbox	v25.2
Input `signal_windows`	`[16, 2400, 1, 1]` (`SCBT`)
Input `time_features`	`[6, 1]` (`CB`)
Output `p_vehicle`	`[1, 1]` scalar $\in [0, 1]$
Simulink sample time	1 s per window
Simulation duration	68 s (1 test file)

References

[1] L. Zhang and H. Chen, "Signal processing based on butterworth filter: Properties, design, and applications," *Highlights in Science, Engineering and Technology*, vol. 97, pp. 112–119, 2024.

[2] A. García-Rodríguez *et al.*, "Hardware-efficient phase demodulation for digital ϕ-otdr receivers with baseband and analytic signal processing," *Sensors*, vol. 25, no. 10, p. 3218, 2025.

[3] A. I. Al-Omari *et al.*, "On control charts based on the generalized poisson model for over-dispersed data," *Electronics*, vol. 14, no. 2, p. 388, 2025.

[4] Y. Zhang *et al.*, "Decoupled pseudo-labeling for semi-supervised monocular 3d object detection," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 14210–14220, 2024.

[5] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 2980–2988, 2017.

[6] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2818–2826, 2016.

[7] L. A. Klein, M. K. Mills, and D. R. P. Gibson, "Traffic detector handbook," *Federal Highway Administration, FHWA-HRT-06-108*, 2006.

[8] P. P. Freitas, R. Ferreira, and S. Cardoso, "Spintronic sensors," *Proceedings of the IEEE*, vol. 104, no. 10, pp. 1894–1918, 2016.

[9] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, Curran Associates, Inc., 2019.

[10] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A time series is worth 64 words: Long-term forecasting with patchtst," *International Conference on Learning Representations (ICLR)*, 2023.

[11] K. Wang and Y. Liu, "Patch-based transformer–graph framework (ptstg) for traffic forecasting in transportation systems," *Applied Sciences*, vol. 15, no. 3, p. 1045, 2025.

[12] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, and T.-Y. Liu, "On layer normalization in the transformer architecture," *Proceedings of the 37th International Conference on Machine Learning (ICML)*, pp. 10524–10533, 2020.

[13] Anonymous, "Between linear and sinusoidal: Rethinking the time encoder in dynamic graph learning," *OpenReview – NeurIPS 2024 Submission*, 2024.

[14] PyTorch Contributors, "Pytorch documentation." `https://pytorch.org/docs/stable/index.html`, 2024. Accessed: 2025-03-12.

[15] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[16] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pp. 1321–1330, 2017.

[17] The MathWorks, Inc., "MATLAB (r2025b)." `https://www.mathworks.com/products/matlab.html`, 2025. Natick, Massachusetts, United States.

[18] The MathWorks, Inc., "Simulink (r2025b)." `https://www.mathworks.com/products/simulink.html`, 2025. Natick, Massachusetts, United States.

[19] The MathWorks, Inc., "Deep learning toolbox (r2025b)." `https://www.mathworks.com/products/deep-learning.html`, 2025. Natick, Massachusetts, United States.

[20] The MathWorks, Inc., "Deep learning toolbox converter for ONNX model format (r2025b)." `https://www.mathworks.com/matlabcentral/fileexchange/67296`, 2025. MATLAB Central File Exchange.

[21] ONNX Community, "ONNX: Open neural network exchange." `https://onnx.ai`, 2024. Accessed: 2025-03-12.